

Big Data Pipeline Monitor

Architektúra distribovanej aplikácie

Jozef Waldhauser

MSWA

2026 léto

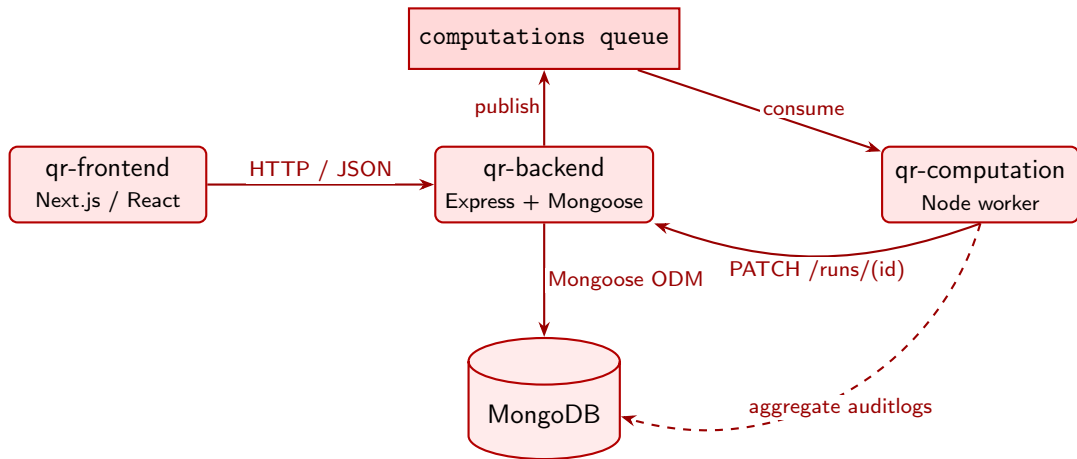
- 1 Prehľad
- 2 Architektúra
- 3 Backend
- 4 Doménový model
- 5 Computation module
- 6 Frontend
- 7 Pipeline
- 8 Stav pipeline
- 9 Designové rozhodnutia

3 komponenty

- Aplikácia eviduje **datasets**, **pipeline**, ich **behy** a **alerty**.
- Spustenie pipeline je **simulovaný výpočet** - reálne sa robí agregácia nad MongoDB.
- Architektonicky som riešenie rozdelil na **tri samostatné služby**, ktoré komunikujú cez REST a frontu správ.

Tri komponenty

qr-frontend → qr-backend → qr-computation-module



- **Synchrónna cesta:** prehliadač → API (CRUD a stav).
- **Asynchrónna cesta:** API → fronta → worker → späť do API.
- Každá služba má **vlastný proces a vlastný port** — nasaditeľné a škálovateľné samostatne.

- Express 5, modulárne routy v `app/`: `datasets`, `pipelines`, `runs`, `alert-rules`, `alerts`.
- Perzistencia cez **Mongoose** (MongoDB) - jeden model na entitu (`Dataset`, `Pipeline`, `JobRun`, `AlertRule`, `Alert`).
- **Validácia** vstupu **Joi** schémami v každom module (`validation.js`).
- **Error handling**: per-route `try/catch`, centrálny 404 a 500 handler v `server.js`.
- Publikácia úloh do RabbitMQ cez `app/rabbitmq.js` (front computations, durable).

Oddelenie zodpovedností

Router robí validáciu a HTTP, model drží schému, business pravidlá sú v handleri (napr. kontrola `active`, stavový prechod).

Doménový model (Mongoose)

- **Dataset** - metadáta (name, owner, type: aggregation|file, aggregation.timeFrom/To).
- **Pipeline** - odkaz na dataset (datasetOid), schedule, active, pipelineVersion, lastStatus, lastRunTime.
- **JobRun** - konkrétny beh: pipelineOid, pipelineVersion, status, časy a processedRecords.
- **AlertRule** - definícia (pipelineOid, reportWhenState).
- **Alert** - vyvolaný záznam (väzba na pipeline + run, acknowledgedAt).

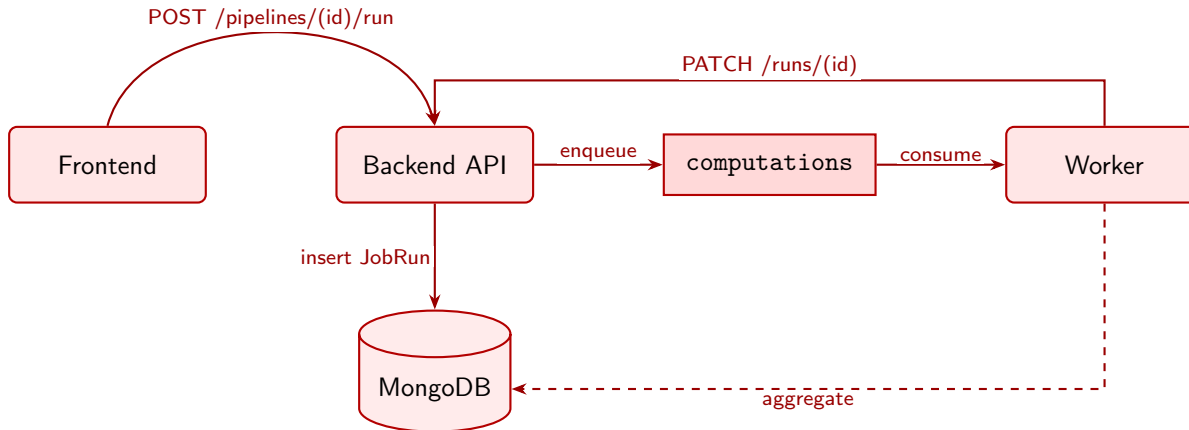
Pravidlo integrity

Pipeline nemôže vzniknúť nad neexistujúcim datasetom - kontroluje sa pred uložením v handleri POST /pipelines.

- Samostatný Node proces s vlastným Express serverom a RabbitMQ consumerom (app/rabbit-consumer).
- Pre každú správu calcStats:
 - 1 PATCH /runs/:id → running (s startTime),
 - 2 agregácia nad MongoDB kolekciou auditlogs v zadanom časovom intervale,
 - 3 uloženie výsledku do computationstats,
 - 4 PATCH /runs/:id → successful / error.
- **Idempotencia** - upsert ComputationJob podľa runId + receiveCount.
- **Odolnosť** - reconnect s exponenciálnym backoffom, prefetch(1), ack/nack.

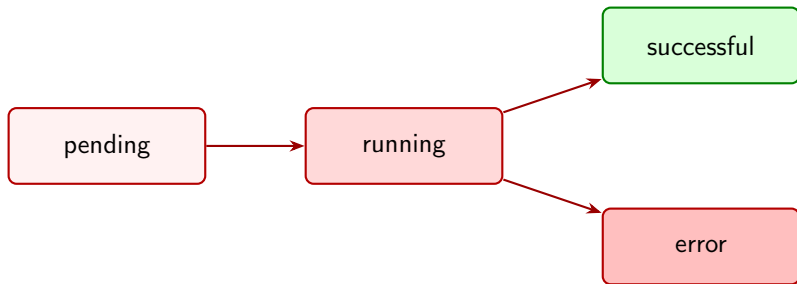
- **Next.js** App Router - React Server Components načítavajú dáta priamo z API počas renderu (`src/lib/api.ts`).
- Stránky: `/` (dashboard), `/datasets`, `/pipelines`, `/pipelines/[id]`, `/runs`, `/runs/[id]`, `/alerts`, `/alert-rules`.
- Klientske komponenty pre akcie a živý stav: `RunPipelineButton`, `RunStatusLive`, `PipelineActiveToggle`, `AcknowledgeAlertButton`.
- **Stavy UI**: *loading* (RSC streaming), *empty* (prázdne zoznamy), *error* (chybové hlášky z `ApiRequestError`).
- Cez `next.config` rewrite na `/blocky-api-same-origin` volania z prehliadača (žiadne CORS).

Komunikácia: spustenie pipeline



- API vráti odpoveď **ihneď** po zaradení do fronty - žiadne dlhé HTTP požiadavky.
- Pri **nedostupnom RabbitMQ** backend run uloží ako error a vráti 503.

Stavový stroj behu a biznis pravidlá



- Povolené prechody sú validované v PATCH /runs/:id - iné kombinácie vrátia 409.
- **Pipeline** sa dá spustiť len ak `active = true` (inak 409).
- **Alert** vznikne podľa `AlertRule.reportWhenState` pri prechode behu do daného stavu.

- **Tri služby namiesto monolitu** - API zostáva responzívne, výpočet je odolný voči pomalým úlohám, dá sa nezávisle škálovať.
- **RabbitMQ ako buffer** - durable fronta, `prefetch(1)`, manuálne `ack/nack` $\Rightarrow$ žiadna strata úloh pri reštarte.
- **MongoDB + Mongoose** - doménové entity sú prirodzene dokumentové, schémy v Mongoose dávajú validáciu a typovú istotu.
- **Joi validácia + jednotné HTTP kódy** - 400 / 404 / 409 / 503 podľa významu.
- **Stavový stroj na serveri** - nedá sa „preskočiť“ priamo z `pending` do `successful`.
- **Next.js RSC** na FE - výpisy sa renderujú serverom z API, klientske komponenty len pre akcie a polling stavu.

Ďakujem za pozornosť.

Otázky?